PANORAMA 2026 · INGENIERÍA DE SOFTWARE

Metodologías *DD

El panorama completo en IA y desarrollo moderno: de TDD a SDD y EDD

TDD · BDD · DDD · SDD · STDD · ASDD · EDD · ODD · IDD



LA IDEA CENTRAL

Cada *DD responde a una sola pregunta: ¿qué artefacto manda sobre el código?

Tests, escenarios, modelos de dominio, especificaciones, evals o contexto. Cambia el artefacto, pero la lógica es la misma: definir qué es “correcto” antes de construir.

Test · Escenario · Modelo de dominio · Spec · Eval · Contexto

EL MAPA

Tres olas de *DD

Ola 1 · 1986–2006

Diseño y calidad

- TDD · Test-Driven
- BDD · Behavior-Driven
- ATDD · Acceptance Test
- DDD · Domain-Driven Design
- FDD · Feature-Driven
- DbC · Design by Contract
- MDD · Model-Driven

Ola 2 · década de 2010

Producto y plataforma

- API-first · Contract-first
- HDD · Hypothesis-Driven
- Data-Driven Development
- CDD · Component-Driven
- TyDD · Type-Driven
- ODD · Observability-Driven

Ola 3 · 2025–2026

Agentes de IA

- Vibe coding
- PDD · Prompt-Driven
- SDD · Spec-Driven
- STDD · Spec-Test-Driven
- ASDD · Agentic Spec-Driven
- EDD · Eval-Driven
- ODD · Organic-Driven
- RDD · Receipt-Driven
- CDD · Context-Driven
- IDD · Intent-Driven

PARTE 01

Las clásicas

El código sigue mandando, pero se verifica contra algo escrito antes: un test, un escenario, un modelo.

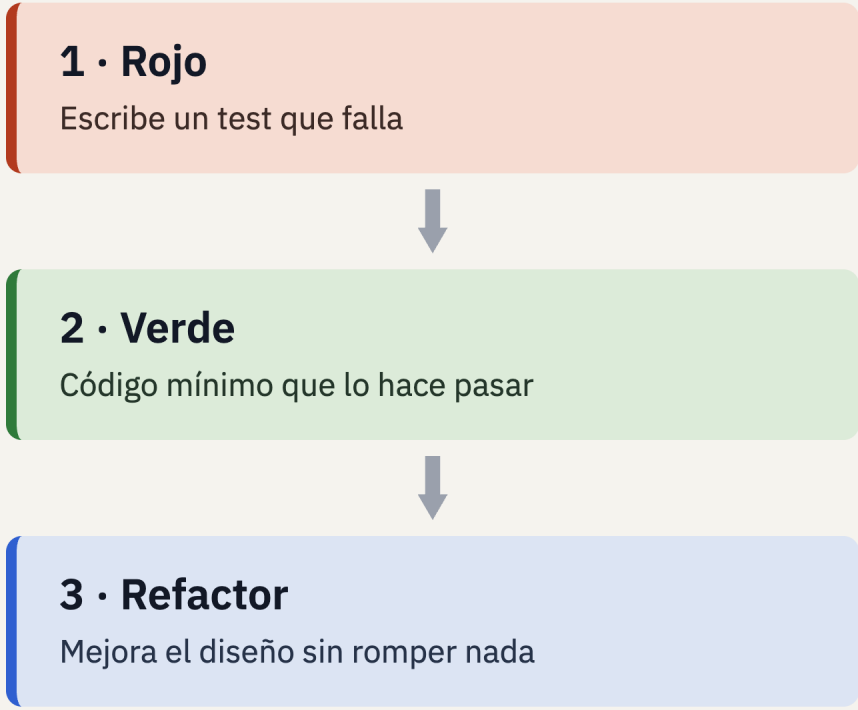
TDD · TEST-DRIVEN DEVELOPMENT

Primero el test, después el código

Kent Beck lo formalizó en *Test-Driven Development: By Example* (2002). Ciclo corto: un test que falla, el mínimo código que lo pasa y una refactorización protegida por el test.

Con agentes vuelve con fuerza: el test es la especificación ejecutable que el agente no puede discutir.

`Artefacto que manda: el test`



BDD · ATDD

Escenarios que entiende el negocio

BDD · Behavior-Driven Development

Dan North, 2006. Describe el comportamiento en lenguaje de negocio con Dado / Cuando / Entonces. Herramientas: Cucumber, SpecFlow, Behave.

ATDD · Acceptance Test-Driven

Negocio, desarrollo y QA (los “tres amigos”) acuerdan los criterios de aceptación como tests antes de construir la funcionalidad.

```
Escenario: reembolso de un pedido reciente
  Dado un pedido pagado hace 3 días
  Cuando el cliente solicita el reembolso
  Entonces el sistema lo aprueba automáticamente
```

Hoy este formato es una spec natural para agentes: legible por personas, verificable por máquinas.

DDD · DOMAIN-DRIVEN DESIGN

Eric Evans, 2003: modelar el dominio antes que el código

Lenguaje ubicuo

Un vocabulario compartido entre negocio y código.

Contextos delimitados

Límites explícitos donde cada modelo tiene sentido.

Agregados

Reglas de consistencia agrupadas en el modelo.

Eventos de dominio

Hechos del negocio que otros contextos escuchan.

CON AGENTES

El lenguaje ubicuo es contexto de alta calidad: un glosario y un mapa de contextos en el repo reducen la ambigüedad.

No confundir con Data-Driven Development, que comparte la sigla.

OLA 1 Y 2

Otras *DD que siguen vigentes

Sigla	Nombre	Qué manda sobre el código	Origen
FDD	Feature-Driven Development	Una lista de features con valor para el cliente	J. De Luca, 1997
DbC	Design by Contract	Precondiciones, postcondiciones e invariantes	B. Meyer, 1986
MDD	Model-Driven Development	Modelos (UML, MDA) que generan código	OMG MDA, 2001
API-first	Contract-first	El contrato OpenAPI, antes que el servicio	Década de 2010
HDD	Hypothesis-Driven Development	Una hipótesis con un experimento medible	Lean Startup, 2010s
CDD	Component-Driven Development	Componentes de UI aislados (Storybook)	c. 2017
TyDD	Type-Driven Development	El sistema de tipos guía la implementación	E. Brady (Idris), 2017

PARTE 02

La era de los agentes

Cuando la IA escribe el código, lo que hay que diseñar es la intención: la spec, las evals y el contexto.

EL CAMBIO DE FONDO

Con agentes que escriben miles de líneas por hora, el cuello de botella ya no es teclear código: es precisar la intención.

Febrero de 2025: Andrej Karpathy bautiza el “vibe coding”. Entre 2025 y 2026 la industria responde con specs, tests y evals que anclan al agente.

VIBE CODING · PDD (PROMPT-DRIVEN DEVELOPMENT)

El prompt como punto de partida

Qué es

Describir el objetivo en lenguaje natural y aceptar el código que devuelve el modelo. El prompt reemplaza al *git clone* como primera línea del proyecto.

Dónde sirve y dónde falla

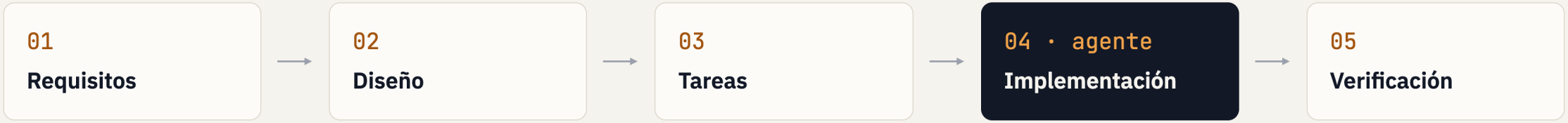
Sirve: prototipos, demos, exploración rápida.

Falla: en producción. El prompt es efímero, el agente olvida decisiones previas y los cambios se vuelven inconsistentes.

SDD · SPEC-DRIVEN DEVELOPMENT

La especificación es la fuente de verdad

Se escribe una spec estructurada antes de que la IA genere código y se mantiene como contrato para humanos y agentes. Si el resultado está mal, se corrige la spec y se regenera.



Herramientas: GitHub Spec Kit · AWS Kiro · OpenSpec · Tessl · BMAD-METHOD

SDD · TRES NIVELES DE ADOPCIÓN

No todo el SDD es igual

Nivel 1

Spec-first

La spec se escribe antes del código y guía la tarea; después puede abandonarse.

Nivel 2

Spec-anchored

La spec se conserva y evoluciona junto al código durante el mantenimiento.

Nivel 3

Spec-as-source

La spec es lo único que editan las personas; el código es un artefacto generado.

Riesgo: una spec subespecificada produce código incorrecto pero “seguro de sí mismo”; el agente rellena los huecos con supuestos plausibles.

Crítica: ¿Waterfall 2.0? La diferencia está en el ciclo: minutos entre spec e implementación, no meses.

STDD · ASDD

Dos extensiones del SDD: más tests y más agentes

STDD · Spec-Test-Driven Development

`Spec → Tests → Código`

Une SDD y TDD en un solo flujo. La spec se aprueba primero; de cada requisito se deriva un test antes de implementar, y el agente no puede dar el trabajo por terminado sin la suite en verde.

Se ve en: skills y plugins para agentes de terminal que fuerzan "un test a la vez" con hooks antes de cada edición o commit.

ASDD · Agentic Spec-Driven Development

`Intención → Spec → Validación → Dominio → Diseño → Plan`

SDD a escala de equipo: varios agentes especializados recorren un pipeline con gates de fase y aprobación humana. Requisitos en sintaxis EARS y bitácora de cada acción del agente.

Lema: la IA propone, las personas gobiernan.

EDD · EVAL-DRIVEN DEVELOPMENT

Si el software no es determinista, las evals son la especificación

Dimensión	TDD	EDD
Pregunta	¿Pasa el test?	¿Supera el umbral de calidad?
Resultado	Binario: verde o rojo	Puntaje continuo (por ejemplo, 0 a 1)
Oráculo	Aserción exacta	Rúbrica, similitud o LLM como juez
Datos	Miles de casos baratos	~100 casos “dorados” bien curados
Gate de release	Suite en verde	Umbrales por dimensión: exactitud, seguridad, tono, latencia

Herramientas: DeepEval · Braintrust · promptfoo · LangSmith · MLflow

ODD (1) · OBSERVABILITY-DRIVEN DEVELOPMENT

Tests en verde no es “funciona”: falta observar

Popularizado por la comunidad de observabilidad (Charity Majors, Honeycomb): instrumentar desde el diseño, no al final.

Con LLMs y agentes resurge: sus salidas no deterministas escapan a los tests unitarios, y la telemetría real alimenta las evals y la próxima spec.

Una telemetría, dos consumidores: dev y ops

01 · Spec e implementación
El ciclo SDD habitual

02 · Instrumentar
OpenTelemetry desde el primer commit

03 · Observar con tráfico real
Trazas, métricas, logs y perfiles

04 · Cerrar el ciclo
Lo observado se vuelve spec o eval

ODD (2) · ORGANIC DRIVEN DEVELOPMENT · ALAN BUSCAGLIA

Lo chico se mantiene chico

Propuesto por Alan Buscaglia (Gentleman Programming) en Gentle-AI, que configura agentes como Claude Code, Cursor, OpenCode o Codex.

Proceso proporcional: un cambio pequeño y entendido no genera artefactos; el trabajo sustancial lleva un único documento de feature que permite retomar.

`Verificar vale más que generar`

Del mismo autor · RDD (Receipt-Driven Development): revisiones independientes según el riesgo, que dejan un "recibo" como evidencia del cambio.

1. **Autorizar:** ¿se pidió un cambio?
2. **Explorar** en proporción al pedido
3. **Resolver** una incertidumbre clave
4. **Clasificar:** ¿pequeño o sustancial?
5. **Registrar** el documento antes de escribir
6. **Implementar** tarea a tarea, con prueba
7. **Cerrar:** resultado verificado y siguiente paso

TDD PARA AGENTES

Los tests como barandas del agente

6,1%

Regresiones del agente sin instrucciones de testing

9,9%

Con instrucciones TDD genéricas en AGENTS.md: un 63% peor

1,8%

Con un mapa de qué tests correr: un 70% mejor

Tests inmutables

El agente implementa contra tests que no puede modificar.

Revisar el rojo

Validar los tests antes de implementar evita tests tautológicos que solo confirman el código.

Fases separadas

Agentes distintos para rojo, verde y refactor, con checkpoints humanos.

CDD · IDD

Context- e Intent-Driven: diseñar lo que el agente sabe

Context-Driven Development

Hace explícitos la intención de producto, los requisitos, las decisiones de arquitectura, las restricciones, los límites de cada tarea y la evidencia de verificación. Busca el contexto mínimo suficiente y con autoridad.

`AGENTS.md · CLAUDE.md · ADRs · skills`

Intent-Driven Development

Paraguas que agrupa context engineering, SDD, harness engineering y loops de agentes. Idea común: estructurar la intención antes de generar. Los límites (lo que la IA no debe hacer) son un artefacto de primera clase.

`intención · límites · evidencia`

Del *prompt engineering* al *context engineering*: el prompt deja de ser el sistema.

PARTE 03

Cómo encajan

Las *DD no compiten: se apilan. Cada una cubre una capa distinta de la pregunta “¿esto es correcto?”.

COMPARATIVA

Qué manda en cada *DD

Metodología	Fuente de verdad	Pregunta que responde
TDD	Tests unitarios	¿Cada unidad hace lo que debe?
BDD / ATDD	Escenarios de negocio	¿El sistema se comporta como espera el negocio?
DDD	Modelo de dominio	¿Código y negocio hablan el mismo idioma?
Vibe / PDD	El prompt	¿Funciona lo que veo ahora mismo?
SDD	Especificación	¿Qué debe construir el agente, y lo construyó?
STDD	Spec + tests derivados	¿Cada requisito de la spec tiene un test en verde?
ASDD	Spec con gates de fase	¿Cada fase del pipeline de agentes fue aprobada?
EDD	Evals y dataset dorado	¿La respuesta es suficientemente buena?
ODD (observ.)	Telemetría real	¿Se comporta bien con tráfico real?
ODD (orgánico)	Un documento de feature	¿El proceso es proporcional al tamaño del cambio?
CDD / IDD	Contexto e intención	¿Qué sabe el agente y qué no debe hacer?

CÓMO SE COMBINAN

Un flujo moderno apila varias *DD



CDD / IDD: el contexto versionado en el repo (AGENTS.md, glosario, ADRs, specs) alimenta cada etapa.

El ciclo se cierra: lo que falla en producción vuelve como nuevo caso de eval, nuevo escenario o nuevo test.

ANTIPATRONES

Cuatro formas de hacer *DD sin hacerla

⚠ Specs subespecificadas

El agente rellena los huecos con supuestos plausibles y produce código incorrecto con total confianza.

⚠ Tests tautológicos

Si el agente escribe los tests leyendo su propia implementación, la cobertura sube pero no prueba nada.

⚠ Evals performativos

Mejoras que desaparecen al re-ejecutar con otro dataset o juez. El harness de evals se trata como código de producción.

⚠ Fatiga de siglas

Rebautizar la práctica sin cambiar el proceso. Si nada se verifica antes de aprobar, no hay *DD.

GUÍA PRÁCTICA

¿Qué *DD usar y cuándo?

Situación	Combinación recomendada
Prototipo o demo desechable	Vibe coding / PDD, sin culpa, y luego se descarta
Trabajo diario con un agente	ODD orgánico: proceso proporcional al tamaño del cambio
Feature productiva hecha con agentes	STDD: spec aprobada, tests derivados e inmutables
Producto con un LLM dentro	EDD: dataset dorado, LLM como juez y gates de release
Dominio de negocio complejo	DDD + BDD como lenguaje común con el negocio
Muchos agentes y equipos en un mismo repo	ASDD + CDD / IDD: pipeline con gates y contexto versionado
Agentes o servicios ya en producción	ODD + EDD: trazas por cada tool call y evals desde datos reales
Sistema crítico o regulado	SDD spec-anchored + Design by Contract + revisión humana en cada gate

EN UNA FRASE

Define qué es “correcto” antes de generar.

La IA abarató escribir código. Lo escaso ahora es la intención precisa y la verificación: ahí viven todas las *DD.

REFERENCIAS

Fuentes

- K. Beck, *Test-Driven Development: By Example* (2002) · E. Evans, *Domain-Driven Design* (2003) · D. North, “Introducing BDD” (2006)
- B. Böckeler (Thoughtworks), “Understanding Spec-Driven Development: Kiro, spec-kit and Tessl” (2025)
- D. B. Piskala, “Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants”, arXiv 2602.00180 (2026)
- Augment Code: What Is Spec-Driven Development? · Built In: Why Spec-Driven Development
- ASDD: Agentic Specification-Driven Development (v5.0, 2026) · STDD Agents
- ODD: Spec-Driven Development Is Only Half the Loop · Forrester: Embrace ODD
- A. Buscaglia (Gentleman Programming), Gentle-AI: Organic Driven Development y Receipt-Driven Development (docs)
- O’Reilly Radar: Evaluation-Driven Development for AI Systems
- Braintrust: What is eval-driven development (2026) · DeepEval: Eval Driven Development
- D. Vaughan: TDAD and the testing revolution (2026) · Test-Driven AI Coding
- intent-driven.dev: Intent-Driven Development · Context Driven Development (Leanpub, 2026)